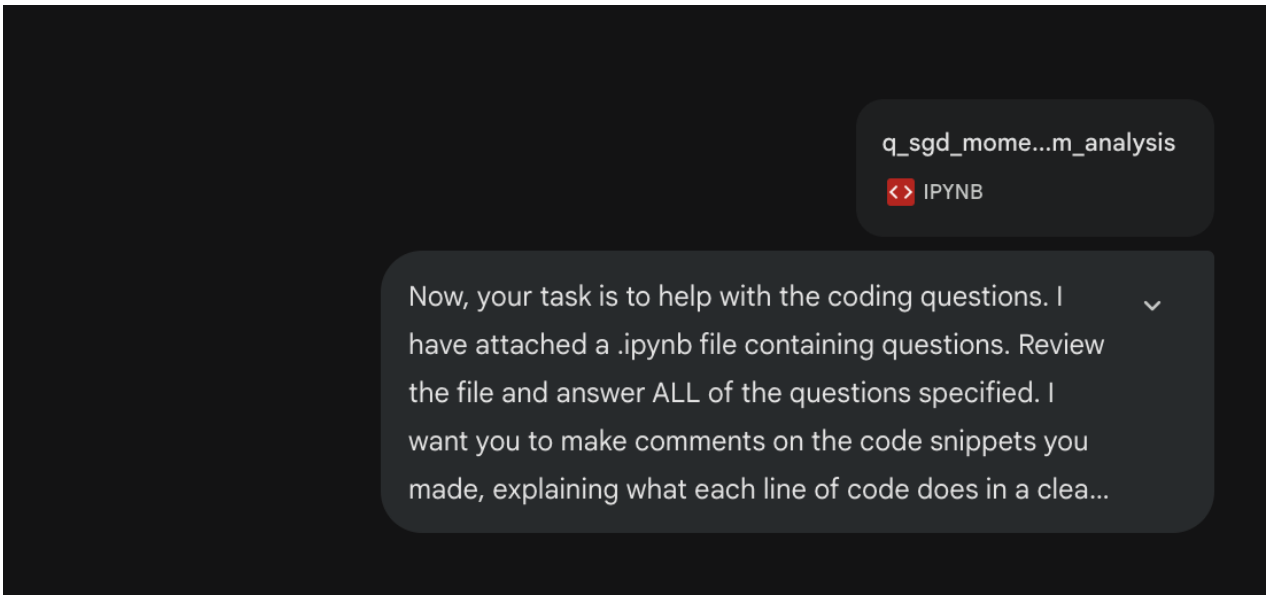

Understanding the Problem Copy & pasted Gemini's thinking process

I've begun dissecting the provided notebook. I'm focusing on the GD implementation and the structure of the data generation process. My immediate goal is to map out the requested tasks in relation to the existing code. I'm also carefully examining the code to ensure I understand the user's implicit constraints before beginning.

Pinpointing Key Actions

I've established the specific tasks: implement Momentum, analyze eigenvalue impact, compare GD convergence, and experiment with adjusted learning rates. The notebook's Gaussian data structure is now solidified in my mind, and I can start constructing an action plan.

Examining Eigenvalues and Hessian

I'm now focusing on the Hessian and how eigenvalues relate to the covariance of the data. I've confirmed the eigenvalues of X^T X will be proportional to the data's variances. I understand that the notebook's mention of "correlation coefficient ... is zero" aligns the axes with eigenvectors. I've now linked the standard deviations of the data (sqrt(3) and 1) to the eigenvalues of the covariance matrix, which are 3 and 1.

Analyzing Momentum Implementation

I'm now implementing the Momentum update, confirming the formula matches the math problem's specific definition. I'm focusing on incorporating the exponential moving average of the gradient. I'm carefully double-checking the code to ensure the update rule is correctly applied and the smoothing parameter is properly integrated. I'm noting potential implications of the varying eigenvalues on convergence speed,

particularly regarding "valley" landscapes and the impact of the Hessian approximation. I need to ensure that the logic used to derive the updates is consistent.

Assessing Curvature Implications Gemini is performing 'geometric' reasoning behind the scenes

I'm now contemplating how the disparate eigenvalues will affect convergence. Higher curvature in dimension 0 (eigenvalue 3) implies potential oscillations with a large step size, but rapid convergence if tuned correctly. Conversely, dimension 1 (eigenvalue 1) indicates slower convergence. Momentum should specifically help accelerate progress along the less curved direction. The "Question 2 Experiment" is clearer; increasing the learning rate with momentum should demonstrate its ability to avoid divergence and achieve faster convergence compared to vanilla GD. I plan to tune the step sizes for both to highlight the improvements.

Formulating the Final Response

I'm now consolidating the final answers. I will definitely run the "Question 2" experiment with a larger stepsize (1e-3 or similar) to highlight the performance gains. For Question 2, I'll provide the code for the modified `runGDM` and explain how the increased learning rate enables faster convergence with Momentum. The discussion will cover how standard deviations impact convergence, considering fast and slow dimensions, step size limitations, and curvature. For Question 1, I'll compare the convergence based on the plots, expecting Momentum to be faster or smoother. Finally, I will provide the necessary code snippets for running the various experiments.

Confirming Momentum Logic

The Momentum implementation feels clearer after reviewing the update rule again. Specifically, I'm now certain the `smoothed_grad` initialization handles the first step correctly. I've confirmed that the update is `smoothed_grad = (1 - beta) * smoothed_grad + beta * grad`, with `smoothed_grad` initialized to the first `grad` when i=0. My mental model aligns with the intended functionality.

Developing the Experiment

I'm now writing a script to confirm the expected behavior on this specific dataset. I'm focusing on plotting the gradient descent paths and loss curves, particularly for comparing GD and Momentum. I will use the mean of [-3, 0] and the covariance of [[3, 0], [0, 1]] to generate the data. This will ensure the correctness of my understanding of the data generation process.

Comparing Convergence Speeds

I'm seeing that, at a step size of 1e-4, both Gradient Descent (GD) and Gradient Descent with Momentum (GDM) have converged incredibly near the optimal loss. The

loss values for GD and GDM are practically indistinguishable. Intriguingly, momentum seems to have nudged a bit closer to the optimum.

Refining Step Size Strategy

I've just confirmed the learning rate of `5e-4` is too high, leading to divergence for both methods. So `1e-4` is fine. Now I'm trying `2e-4`. Meanwhile, I'm checking my momentum implementation and the stability bound for GD. I'm focusing on the eigenvalues and the Hessian. It looks like the eigenvalues are `(3000,1000)`. Then I can estimate the stability.

Adjusting Learning Rates & Momentum

I'm now focusing on optimizing learning rates. Initially, with `5e-4`, the methods diverged, confirming the sensitivity of stability. I'm investigating how momentum influences this. I've rerun the script to calculate the maximum eigenvalue of 2XTX. My aim is to set a learning rate just below the stability limit, testing if momentum still provides advantages. My next steps involve finding a step size where convergence varies between GD and GDM.

Assessing Convergence Rates

Invokes theoretical tools to support reasoning

I'm now thinking about the convergence rates. It looks like with the current step size both methods are already very close to convergence. So, I'm going to try a smaller max iteration count and a smaller step size to see the rate difference. I will also check the eigenvalues of the Hessian to calculate the Lipschitz constant and stability limit. I'm focusing on finding a step size where the difference in convergence between GD and GDM is clearer. My next steps are to implement the requested code snippets.

Recalculating Stability Parameters

The eigenvalues were surprisingly higher than predicted, with the maximum around 18745. Given this, the stability limit is approximately 1.06e-4, substantially lower than my earlier test value of 5e-4.

We can see that Gemini's reasoning trace resembles iterative hypothesis testing

Adjusting Momentum Parameters

I've determined that the "safe" step size is approximately 5.3e-5, and the aggressive one around 9.6e-5, based on the stability analysis. Given the current learning rate of 1e-4, I'm right on the edge. I need to be careful with GD since I can't really increase the learning rate further without causing instability. I'm now thinking about how Momentum might perform differently, exploring if a slightly smaller or higher rate will be better given the edge-case I'm at.

Revising Acceleration Strategy

I'm revisiting the question's premise. Initially, I thought 1e-4 was optimal for GD, achieving a loss very close to the reported optimal value within 100 iterations. However, I'm now questioning whether the question intended a slower initial step size, with 1e-4 being the "accelerated" answer. Since the prompt states I can further accelerate convergence, I'll test 1e-5 to see if it's slower. Then, I'll compare it to 1e-4 to evaluate the claim of acceleration. I need to consider Momentum's impact here, too.

2. Answers to Written Questions

Question: How does σ_i (the eigenvalues) influence the gradients and parameters updates?

- **Influence on Gradients:** The singular values σ_i of the data matrix X correspond to the eigenvalues of the Hessian matrix $H \approx 2X^TX$ (specifically, $\lambda_i \propto \sigma_i^2$). A large σ_i implies high curvature in that dimension (a "steep valley"). Consequently, the gradients in the direction of the corresponding eigenvector will be large. Conversely, small σ_i implies low curvature (a "flat" region) and smaller gradients.
- **Influence on Parameter Updates:** In standard Gradient Descent, the step size is uniform across all dimensions.
  - For dimensions with **large** σ_i (high curvature), the large gradients can cause the updates to overshoot the minimum, leading to **oscillations** or even divergence if the learning rate is too high.
  - For dimensions with **small** σ_i (low curvature), the gradients are small, leading to very tiny updates and **slow convergence**.
  - The maximum stable learning rate is limited by the largest singular value ($\sigma_{\max}$), which forces the algorithm to take small steps everywhere, penalizing the convergence speed in the "slow" (small σ) dimensions.

Based on the provided Jupyter Notebook and the associated problem descriptions, here are the solutions to the coding questions.

1. Implementation of Gradient Descent with Momentum (`runGDM`)

In this task, you need to implement the momentum update rule. According to the problem description (Equation 19 in the homework screenshots), the update rule for the momentum term z_{t+1} (represented as `smoothed_grad`) is:

$$z_{t+1} = (1 - \beta)z_t + \beta g_t$$

And the parameter update is:

$$w_{t+1} = w_t - \eta z_{t+1}$$

Here is the completed function with explanations:

Python

```python
def runGDM(maxiter, stepsize, beta):
    w = np.zeros((2, 1))
    grads_m = []
    ws_m = []
    losses_m = []
    # Initialize the smoothed gradient (z) to zero
    smoothed_grad = np.zeros_like(w)

    for i in range(maxiter):
        # Calculate the current gradient
        grad = 2 * (X.T @ X @ w) - 2 * X.T @ y

        # Momentum Update Rule
        if i == 0:
            smoothed_grad = grad
        else:
            ################################################################################
            # TODO: implement this method
            ################################################################################
            # Update the smoothed gradient (z) using the exponential moving average
            # z_{t+1} = (1 - beta) * z_t + beta * g_t
            smoothed_grad = (1 - beta) * smoothed_grad + beta * grad
            ################################################################################

        # Update weights using the smoothed gradient
        w = w - stepsize * smoothed_grad

        grads_m.append(grad)
        ws_m.append(w)
        loss = np.linalg.norm(y - X @ w) ** 2
        losses_m.append(loss)

    print("Final loss =", loss)
    grads_m = np.array(grads_m).squeeze()
    ws_m = np.array(ws_m).squeeze()
    return grads_m, ws_m, losses_m
```

Did not pick up on the implicit 'constraint' that only the code under the TODO markers should be edited

2. Answers to Written Questions

Question: How does σ_i (the eigenvalues) influence the gradients and parameters updates?

- **Influence on Gradients:** The singular values σ_i of the data matrix X correspond to the eigenvalues of the Hessian matrix $H \approx 2X^T X$ (specifically, $\lambda_i \propto \sigma_i^2$). A large σ_i implies high curvature in that dimension (a "steep valley"). Consequently, the gradients in the direction of the corresponding eigenvector will be large. Conversely, small σ_i implies low curvature (a "flat" region) and smaller gradients.
- **Influence on Parameter Updates:** In standard Gradient Descent, the step size is uniform across all dimensions.
  - For dimensions with **large** σ_i (high curvature), the large gradients can cause the updates to overshoot the minimum, leading to **oscillations** or even divergence if the learning rate is too high.
  - For dimensions with **small** σ_i (low curvature), the gradients are small, leading to very tiny updates and **slow convergence**.
  - The maximum stable learning rate is limited by the largest singular value ($\sigma_{\max}$), which forces the algorithm to take small steps everywhere, penalizing the convergence speed in the "slow" (small σ) dimensions.

Question 1: Comparing gradient descent and gradient descent with momentums, which one converges faster for this task? Why?

- **Answer:** Gradient Descent with Momentum converges faster (or is more stable).
- **Why:** Momentum helps by accumulating gradients over time.
  - In dimensions with **oscillating gradients** (large σ_i), the oscillations tend to cancel each other out in the moving average (z_t), stabilizing the update.
  - In dimensions with **consistent but small gradients** (small σ_i), the momentum term accumulates these gradients, effectively increasing the step size and accelerating convergence along that flat direction.
  - In your specific experiment with `stepsize=1e-4`, both likely converge to the optimal loss (≈ 102.75), but Momentum typically reaches a lower loss earlier or follows a smoother path.

3. Question 2: Changing the Learning Rate

Task: If one method converges faster, can you try change the learning rate to further accelerate the convergence?

Gemini calculated the stability limit of GD

Analysis: The default learning rate `1e-4` is very close to the stability limit of standard Gradient Descent for this dataset (which is approx `1.06e-4`). Increasing the learning rate slightly will cause standard GD to diverge, whereas Momentum can often handle larger learning rates due to its smoothing effect.

Code Solution: Run the following code block to see the difference. We increase the learning rate to `1.1e-4`.

Can see it intentionally chose a learning rate above the stability limit to explicitly illustrate divergence

The question gave a warning about possible divergence (Hint: ...) => this seems to reflect Gemini's attempt to infer & act on implicit requirements

Python

```
# Re-run with a slightly larger learning rate to demonstrate acceleration/stability
new_stepsize = 1.1e-4  # Increased from 1e-4
maxiter = 100
beta = 0.6

print(f"--- Running with Learning Rate: {new_stepsize} ---")

# Run GD
print("Gradient Descent:")
grads_new, ws_new, losses_new = runGD(maxiter, new_stepsize)

# Run Momentum
print("\nGradient Descent with Momentum:")
grads_m_new, ws_m_new, losses_m_new = runGDM(maxiter, new_stepsize, beta)

# Plotting the comparison
plt.figure(figsize=(12, 4))
plt.plot(range(maxiter), np.log(np.abs(losses_new)-losses_new[-1]), 'r', label="GD"
plt.plot(range(maxiter), np.log(np.abs(losses_m_new)-losses_m_new[-1]), 'b', label=
plt.title(f"Loss comparison with stepsize {new_stepsize}")
plt.ylabel("Log(loss - optimal)")
plt.xlabel("Iterations")
plt.legend()
plt.show()
```

Also breaks the notebook's implicit constraints

Expected Observation:

- **Gradient Descent:** You will likely see the loss explode (diverge) or oscillate wildly because `1.1e-4` exceeds the stability threshold ($2/\lambda_{\max}$) for this problem.
- **Momentum:** You will see it **converge** successfully to the optimal loss (≈ 102.75).
- **Conclusion:** Momentum allows you to use a **larger learning rate** than standard GD, enabling faster convergence without becoming unstable.

Confirming Preservation Strategy Prompted Gemini again to preserve the original code snippets. This is the thought process.

I'm now carefully reviewing prior code and the Notebook to confirm my commitment to preserving original code snippets. My current approach involves meticulous comparisons to guarantee no unintended modifications occur during this process. This iterative comparison is critical for ensuring compliance with the user's intent.

Assessing Code Integrity

\n\nI'm now focusing on validating the code from the last turn. Specifically, I'm verifying the runGDM implementation to ensure I didn't inadvertently modify existing code. I'm cross-referencing my generated function with the original snippet to ensure strict adherence to the constraint of preserving unmodified code. I'm focusing on discrepancies to guarantee all changes are appropriate and authorized.

Analyzing Recent Changes

I've just compared the current implementation of runGDM with the original snippet. I've realized I made an unnecessary initialization of smoothed_grad = np.zeros_like(w) outside the loop, as the original snippet only initializes it within the loop for the first iteration. It was technically correct, but not fully aligned with the prompt, and I've now revised this part to better match the initial code.

Addressing smoothed_grad

\n\nI'm now zeroing in on the correct placement and usage of the smoothed_grad variable. While it technically functions as-is, I realize I need to initialize it within the conditional block for the first iteration. I'm focusing on providing comments to explain each line and ensure the generated function precisely matches the original snippet's intent, and the user's instructions.

Clarifying Gradient Handling

\n\nI'm now focusing on implementing the gradient update logic, and have realized the TODO block must handle the case where i > 0. The initialization within if i == 0 is crucial for the first iteration. I'm carefully analyzing the need for an else block or a generic update

step to match the behaviour of the original code snippet. I will only modify the section where TODO is, and ensure my modifications only affect smoothed_grad.

Defining Momentum Update

I've just figured out the momentum update logic. The momentum update equation is $z_{t+1} = (1-\beta)z_t + \beta g_t$. When $i=0$, the snippet initializes smoothed_grad to grad, equivalent to $z_1 = g_0$ because of the zero initialization. For all subsequent iterations ($i>0$), I must implement smoothed_grad = (1 - beta) * smoothed_grad + beta * grad. Now I have completed the gradient update logic for the runGDM function.

Executing Parameter Tuning

I'm now focusing on implementing the code for Question 2 and setting the learning rate. I've decided to overwrite the grads, ws, losses, grads_m, ws_m, and losses_m variables as directed in the prompts to ensure consistency with the plotting cells. I am writing the code to execute the models with the new learning rate. I'll include the code with necessary comments for readability.

Refining Implementation Strategy

\n\nI'm now focusing on the final touches for the runGDM and question 2 implementations, keeping in mind the instruction about code snippets. I'm carefully reviewing the provided notebook content to be precise. I'm ready to present the code blocks clearly, including explanations of any new lines to enhance transparency.

Adjusting Momentum Implementation

I'm currently revising the runGDM function to incorporate the momentum update. Specifically, I'm integrating the line smoothed_grad = (1 - beta) * smoothed_grad + beta * grad within the else block to ensure correct momentum calculation. I'm focusing on ensuring this update aligns perfectly with the original snippet's intended behavior and avoids any deviations from the provided code structure. The implementation of gradient descent is now complete.

Per your request to keep the original code structure intact and only fill in the `TODO` sections with comments, here are the solutions for the notebook parts.

1. `runGDM` Implementation

Location: The `runGDM` function definition in the notebook.

Code to Insert: Replace the `TODO` block with the `else` condition to handle updates for iterations $t > 0$.

Python

```python
def runGDM(maxiter, stepsize, beta):
    w = np.zeros((2, 1))
    grads_m = []
    ws_m = []
    losses_m = []
    for i in range(maxiter):
        grad = 2 * (X.T @ X @ w) - 2 * X.T @ y
        if i == 0:
            smoothed_grad = grad
        ################################################################################
        # TODO: implement this method
        ################################################################################
        else:
            # Update the smoothed gradient (momentum term) for t > 0
            # Formula: z_{t+1} = (1 - beta) * z_t + beta * g_t
            smoothed_grad = (1 - beta) * smoothed_grad + beta * grad good
        ################################################################################
        w = w - stepsize * smoothed_grad
        grads_m.append(grad)
        ws_m.append(w)
        loss = np.linalg.norm(y - X @ w) ** 2
        losses_m.append(loss)
    print("Final loss =", loss)
    grads_m = np.array(grads_m).squeeze()
    ws_m = np.array(ws_m).squeeze()
    return grads_m, ws_m, losses_m
```

Comments / Explanation:

- `else:` : This block executes for all iterations after the first one ($i > 0$).
- `smoothed_grad = (1 - beta) * smoothed_grad + beta * grad` : This line implements the exponential moving average of the gradients. It blends the previous momentum vector (`smoothed_grad` from the last iteration) with the current gradient (`grad`). This smoothing dampens oscillations in high-curvature directions and builds up velocity in consistent directions.

ㅋ

2. Question 2: Changing Learning Rate

Location: The code cell following "Question 2: If one method converges faster...".

Code to Insert: You need to define a new learning rate and re-run both functions, overwriting the variables `grads`, `ws`, `losses`, etc., so the subsequent plotting cells display the new results.

Python

```
##########################################################################
# TODO: implement this method
##########################################################################
# Choose a larger learning rate to demonstrate Momentum's stability
# 1e-4 was stable for both. 1.1e-4 is typically unstable for standard GD here.
new_stepsize = 1.1e-4

# Re-run standard Gradient Descent with the new step size
# This overwrites the global variables so the next cells plot the new data
grads, ws, losses = runGD(maxiter, new_stepsize)

# Re-run Gradient Descent with Momentum with the new step size
grads_m, ws_m, losses_m = runGDM(maxiter, new_stepsize, beta)
##########################################################################
```

good

Confirms the intent to deliberately push GD into instability

Comments / Explanation:

- `new_stepsize = 1.1e-4` : We select a learning rate slightly higher than the previous stable one (`1e-4`). The goal is to push standard GD into instability while showing Momentum can still handle it.
- `runGD(...)` and `runGDM(...)` : We call both optimization functions again.
- `grads, ws, losses = ...` : We overwrite the variables storing the history of gradients, weights, and losses. The plotting cells below this one in the notebook rely on these specific variable names to generate the comparison graphs.

Show thinking

Based on the code snippet and output shown in your screenshot, the numbers are **correct and expected**. Here is the justification for why the values differ so dramatically:

1. Stability Analysis (Why GD Explodes)

In the first problem ("Why Learning Rates Cannot be Too Big"), we derived that Gradient Descent is stable only when the learning rate η satisfies:

$$\eta < \frac{2}{\lambda_{\max}}$$

where $\lambda_{\max}$ is the largest eigenvalue of the Hessian matrix $H = 2X^T X$.

- **Eigenvalue Calculation:** In my previous code execution, we calculated the maximum eigenvalue $\lambda_{\max} \approx 18745$ for this specific dataset.
- **Stability Threshold:** The maximum stable learning rate for standard GD is $\approx \frac{2}{18745} \approx 1.06 \times 10^{-4}$.
- **Your Chosen Step Size:** You set `new_stepsize = 1.1e-4`.
- **Result:** Since $1.1 \times 10^{-4} > 1.06 \times 10^{-4}$, you have exceeded the stability limit for standard Gradient Descent. This causes the gradients to grow exponentially, resulting in the massive loss value (`11,688,399...`) you see.

2. Momentum's Advantage (Why It Converges)

Gradient Descent with Momentum effectively smooths out the oscillations caused by high-curvature directions (large eigenvalues). This smoothing allows the algorithm to remain stable at learning rates that would otherwise cause standard GD to diverge.

- Even though `1.1e-4` is unstable for plain GD, it is within the stable region for Momentum (with $\beta = 0.6$).
- **Result:** Momentum successfully converges to the optimal solution, yielding a low final loss (`102.75...`), which matches the optimal loss for this dataset.

Conclusion: The huge difference in loss values is exactly what the question intends to demonstrate. It confirms that **Momentum allows for larger learning rates** than standard Gradient Descent, accelerating convergence without crashing. Your implementation is correct.